Confluent Coding — Consolidated Questions & Answers

One file: every question → one-line idea → clean Python you can retype in an interview → complexity. ⭐ = seen in Confluent interviews.

Pattern index

Pattern	Questions
Hash map + cache / TTL design	LRU Cache ⭐, LFU Cache, TimeMap ⭐, Time-Limited Cache ⭐, Sliding-Window TTL Cache ⭐, Max/Avg KV Store, Auth Manager, Token Manager, Logger, SensorHealth, RandomizedSet
Text search	Document + Phrase Search (inverted index) ⭐
Two pointers / sliding window	Count events in (t−W, t], Min Start Value
Heaps & greedy	Kth Largest, Top K Frequent, Median Finder, Eliminate Monsters, Min-Health Gaming
Backtracking	Sudoku Validate ⭐ + Solve ⭐, Word Search, Combination Sum I/II/III
Trie	WordDictionary, Word Search II
Graph	Course Schedule I/II, Monster Min-Cost (Dijkstra)
DP / memo	Word Break, Min Days (oranges), Combination Sum IV, Value Meals ⭐, Subset Sum, Warehouse Loading, Wildcard Matching
Binary search on answer	Hens & Grains, Min Initial Strength
Intervals	Meeting Rooms II/III, Meeting Scheduler
Linked lists	Merge k Sorted Lists
Streaming / files	tail -n ⭐
Design / matching	Function Signature Matching ⭐, RandomQueue ADT

Hash Map + Cache / TTL Design

LRU Cache (LC 146) ⭐

Question: `get(key)` → value or -1; `put(key, value)` inserts/updates, evicting the least recently used key at capacity. Both O(1). **Idea:** `OrderedDict` keeps insertion order — touch = move to end, evict = pop from front.

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.cap, self.d = capacity, OrderedDict()

    def get(self, key):
        if key not in self.d:
            return -1
        self.d.move_to_end(key)
        return self.d[key]

    def put(self, key, value):
        if key in self.d:
            self.d.move_to_end(key)
        self.d[key] = value
        if len(self.d) > self.cap:
            self.d.popitem(last=False)
```

Complexity: O(1) per op, O(capacity) space. (From-scratch version: hash map + doubly linked list with dummy head/tail.)

LFU Cache (LC 460)

Question: Like LRU, but evict the least *frequently* used key (ties → least recently used). O(1) ops. **Idea:** key→value, key→freq, freq→OrderedDict of keys; track `min_freq`.

```
from collections import defaultdict, OrderedDict

class LFUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.min_freq = 0
        self.kv = {}
        self.kf = {}
        self.fk = defaultdict(OrderedDict)

    def _touch(self, key):
        f = self.kf[key]
        del self.fk[f][key]
        if not self.fk[f]:
            del self.fk[f]
            if self.min_freq == f:
```

```
                self.min_freq += 1
        self.kf[key] = f + 1
        self.fk[f + 1][key] = None

    def get(self, key):
        if key not in self.kv:
            return -1
        self._touch(key)
        return self.kv[key]

    def put(self, key, value):
        if self.cap == 0:
            return
        if key in self.kv:
            self.kv[key] = value
            self._touch(key)
            return
        if len(self.kv) == self.cap:
            old, _ = self.fk[self.min_freq].popitem(last=False)
            if not self.fk[self.min_freq]:
                del self.fk[self.min_freq]
            del self.kv[old]
            del self.kf[old]
        self.kv[key] = value
        self.kf[key] = 1
        self.fk[1][key] = None
        self.min_freq = 1
```

Complexity: O(1) per op, O(capacity) space.

TimeMap — Time Based Key-Value Store (LC 981) ⭐

Question: `set(key, value, timestamp)`; `get(key, timestamp)` returns the value with the largest timestamp ≤ given, else `""`. Set timestamps are strictly increasing. **Idea:** Per key keep parallel sorted lists; `bisect_right` for get.

```
from collections import defaultdict
import bisect

class TimeMap:
    def __init__(self):
        self.ts = defaultdict(list)
        self.vals = defaultdict(list)

    def set(self, key, value, timestamp):
        self.ts[key].append(timestamp)
        self.vals[key].append(value)

    def get(self, key, timestamp):
        if key not in self.ts:
            return ""
```

```
        i = bisect.bisect_right(self.ts[key], timestamp)
        return self.vals[key][i - 1] if i else ""
```

Complexity: `set` O(1), `get` O(log n).

Cache With Time Limit (LC 2622) ⭐

Question: `set(key, value, duration)` returns True if an unexpired key existed (overwrite it); `get(key)` → value or -1; `count()` → unexpired keys. **Idea:** Store `(value, expiry)`; check expiry lazily — no timers needed in Python.

```
import time

class TimeLimitedCache:
    def __init__(self):
        self.d = {}

    def set(self, key, value, duration_ms):
        now = time.monotonic()
        existed = key in self.d and self.d[key][1] > now
        self.d[key] = (value, now + duration_ms / 1000)
        return existed

    def get(self, key):
        item = self.d.get(key)
        if item and item[1] > time.monotonic():
            return item[0]
        self.d.pop(key, None)
        return -1

    def count(self):
        now = time.monotonic()
        self.d = {k: v for k, v in self.d.items() if v[1] > now}
        return len(self.d)
```

Complexity: `set` / `get` O(1), `count` O(n). (JS version uses `setTimeout` per key instead.)

Sliding-Window TTL Cache + Average ⭐ (Confluent qualifier round)

Question: `put(key, value, t)`, `get(key, t)` (None if expired), `average(t)` over values in the last `window` seconds. E.g. window=1h: put(A,10)@0:00, put(B,20)@0:10 → avg@0:30 = 15; avg@1:05 = 20 (A expired); get(B)@1:08 = 20. **Idea:** Dict key→(value, ts) + deque of (ts, key) in insertion order + running sum/count; evict stale from the left, lazily skipping overwritten keys via timestamp check.

```
from collections import deque
```

```
class SlidingWindowCache:
    def __init__(self, window=3600):
        self.w = window
        self.d = {}
        self.q = deque()
        self.sum = 0.0
        self.n = 0

    def _evict(self, now):
        while self.q and now - self.q[0][0] >= self.w:
            ts, key = self.q.popleft()
            if self.d.get(key, (None, None))[1] == ts:
                v, _ = self.d.pop(key)
                self.sum -= v
                self.n -= 1

    def put(self, key, val, now):
        self._evict(now)
        if key in self.d:
            self.sum -= self.d[key][0]
            self.n -= 1
        self.d[key] = (val, now)
        self.q.append((now, key))
        self.sum += val
        self.n += 1

    def get(self, key, now):
        self._evict(now)
        return self.d[key][0] if key in self.d else None

    def average(self, now):
        self._evict(now)
        return self.sum / self.n if self.n else 0.0
```

Complexity: O(1) amortized per op, O(n) space. (Alt: hash map + doubly linked list ordered by time.)

Max/Average KV Store (get_max under updates)

Question: `put/get/get_average` plus `get_max`. The catch: updating the key holding the max must reveal the *new* max. **Idea:** Running sum for average; max-heap of `(-value, key)` with lazy deletion — pop stale tops until the heap top matches the dict.

```
import heapq

class MaxAverageKVStore:
    def __init__(self):
        self.d = {}
        self.heap = []
        self.sum = 0.0
        self.n = 0
```

```
    def put(self, key, val):
        if key in self.d:
            self.sum += val - self.d[key]
        else:
            self.sum += val
            self.n += 1
        self.d[key] = val
        heapq.heappush(self.heap, (-val, key))

    def get(self, key):
        return self.d[key]

    def get_average(self):
        return self.sum / self.n

    def get_max(self):
        while self.heap:
            neg_v, key = self.heap[0]
            if self.d.get(key) == -neg_v:
                return -neg_v
            heapq.heappop(self.heap)
```

Complexity: `put` O(log n), `get` / `get_average` O(1), `get_max` amortized O(1). (Alt: value-ordered doubly linked list for strict O(1) max.)

Authentication Manager (LC 1797)

Question: Tokens live `ttl` seconds. `generate(id, t)`, `renew(id, t)` only if unexpired, `countUnexpiredTokens(t)`. **Idea:** Map token → expiry; alive means `expiry > now`.

```
class AuthenticationManager:
    def __init__(self, timeToLive):
        self.ttl = timeToLive
        self.exp = {}

    def generate(self, tokenId, currentTime):
        self.exp[tokenId] = currentTime + self.ttl

    def renew(self, tokenId, currentTime):
        if self.exp.get(tokenId, 0) > currentTime:
            self.exp[tokenId] = currentTime + self.ttl

    def countUnexpiredTokens(self, currentTime):
        return sum(e > currentTime for e in self.exp.values())
```

Complexity: `generate` / `renew` O(1), `count` O(n).

Token Manager (generate / renew / list by create_time)

Question: `generate(token, now)`, `renew(token, now)` if unexpired, `active(now)` → active token ids ordered by `create_time` (ties → token id). **Idea:** Dict for O(1) lookup + sorted list of `(create_time, token)` via `bisect.insort`; filter expired at query time.

```
import bisect

class TokenManager:
    def __init__(self, ttl):
        self.ttl = ttl
        self.d = {}
        self.order = []

    def generate(self, token, now):
        if token in self.d:
            return False
        self.d[token] = [now, now + self.ttl]
        bisect.insort(self.order, (now, token))
        return True

    def renew(self, token, now):
        rec = self.d.get(token)
        if not rec or rec[1] <= now:
            return False
        rec[1] = now + self.ttl
        return True

    def active(self, now):
        return [t for _, t in self.order
                if t in self.d and self.d[t][1] > now]
```

Complexity: `generate` O(n) (list insert), `renew` O(1), `active` O(n).

Logger Rate Limiter

Question: `shouldPrintMessage(timestamp, message)` → True unless the same message was printed < 10s ago. Per-message cooldown; timestamps chronological. **Idea:** Dict message → last printed time; allow if `now - last >= 10`.

```
class Logger:
    def __init__(self):
        self.last = {}

    def shouldPrintMessage(self, timestamp, message):
        if timestamp - self.last.get(message, -10) >= 10:
            self.last[message] = timestamp
            return True
        return False
```

Complexity: O(1) per call, O(m) space. (Memory follow-up: queue+set to evict entries older than the window.)

SensorHealth — silent sensor detector

Question: Record ping timestamps per sensor; `is_stable(sensor, T)` → STABLE if a ping exists in `[T-K, T]`, else UNSTABLE. Pings may arrive out of order. **Idea:** Dict sensor → sorted list via `bisect.insort`; `bisect_right(T)` finds the latest ping ≤ T, check it's within K.

```
import bisect
from collections import defaultdict

class SensorHealth:
    def __init__(self, k=10):
        self.k = k
        self.pings = defaultdict(list)

    def record_ping(self, sensor, ts):
        bisect.insort(self.pings[sensor], ts)

    def is_stable(self, sensor, t):
        hist = self.pings.get(sensor)
        if not hist:
            return "UNSTABLE"
        i = bisect.bisect_right(hist, t)
        if i == 0:
            return "UNSTABLE"
        return "STABLE" if t - hist[i - 1] <= self.k else "UNSTABLE"
```

Complexity: record O(n) insert (O(log n) find), query O(log n). (If pings arrive in order, just store the latest timestamp per sensor: O(1).)

RandomizedSet (LC 380)

Question: `insert`, `remove`, `getRandom` (uniform) — all average O(1). **Idea:** List for random access + dict value→index; remove = swap with last, pop.

```
import random

class RandomizedSet:
    def __init__(self):
        self.a = []
        self.pos = {}

    def insert(self, val):
        if val in self.pos:
            return False
        self.pos[val] = len(self.a)
        self.a.append(val)
```

```
        return True

    def remove(self, val):
        if val not in self.pos:
            return False
        i, last = self.pos[val], self.a[-1]
        self.a[i] = last
        self.pos[last] = i
        self.a.pop()
        del self.pos[val]
        return True

    def getRandom(self):
        return random.choice(self.a)
```

Complexity: O(1) average per op, O(n) space.

Text Search

Document + Phrase Search (inverted index) ⭐

Question: Given docs (id → text), `search("cloud")` → doc ids containing the word; `search("cloud monitoring")` → doc ids where the words appear consecutively. Scale: 1M+ docs.
Idea: Inverted index term → {doc_id → [positions]}; candidates = docs containing every term; verify consecutive positions.

```
import re
from collections import defaultdict

class PhraseSearchIndex:
    def __init__(self):
        self.index = defaultdict(lambda: defaultdict(list))

    @staticmethod
    def _tok(text):
        return re.findall(r"\w+", text.lower())

    def add_document(self, doc_id, text):
        for pos, term in enumerate(self._tok(text)):
            self.index[term][doc_id].append(pos)

    def search(self, phrase):
        terms = self._tok(phrase)
        if not terms or any(t not in self.index for t in terms):
            return []
        postings = [self.index[t] for t in terms]
        cands = set(min(postings, key=len))
        for p in postings:
```

```
            cands &= p.keys()
        res = []
        for doc in cands:
            later = [set(p[doc]) for p in postings[1:]]
            for s in postings[0][doc]:
                if all(s + i + 1 in later[i] for i in range(len(later))):
                    res.append(doc)
                    break
        return sorted(res)
```

Complexity: Build O(total tokens); word query O(1)-ish, phrase O(candidates × phrase length). (Follow-ups: AND/OR/bracket queries = set ops on postings.)

Two Pointers / Sliding Window

Count events in (t−W, t] — O(n)

Question: Sorted timestamps; for each event count events in `(t−W, t]`. E.g. `[1,2,4,7]`, W=3 → `[1,2,2,1]`. **Idea:** Left pointer = first index with `ts[right] − ts[left] < W`; count = `right − left + 1`.

```
def count_events(ts, w):
    res, left = [], 0
    for right, t in enumerate(ts):
        while t - ts[left] >= w:
            left += 1
        res.append(right - left + 1)
    return res
```

Complexity: O(n) time, O(n) space for output.

Min Start Value (LC 1413)

Question: Smallest positive `x` so every running total of `x + nums` stays > 0. E.g. `[-3,2,-3,4,2]` → 5. **Idea:** Running total = `x + prefix`; need `x + min_prefix ≥ 1` → `x = max(1, 1 − min_prefix)`.

```
def minStartValue(nums):
    prefix = min_prefix = 0
    for v in nums:
        prefix += v
        min_prefix = min(min_prefix, prefix)
    return max(1, 1 - min_prefix)
```

Complexity: O(n) time, O(1) space.

Heaps & Greedy

Kth Largest (LC 215)

Question: k-th largest in `nums`. **Idea:** Min-heap of the k largest seen; root is the answer.

```
import heapq

def findKthLargest(nums, k):
    heap = nums[:k]
    heapq.heapify(heap)
    for x in nums[k:]:
        if x > heap[0]:
            heapq.heapreplace(heap, x)
    return heap[0]
```

Complexity: O(n log k) time, O(k) space.

Top K Frequent (LC 347)

Question: k most frequent elements. **Idea:** Bucket by frequency — O(n), beats a heap.

```
from collections import Counter

def topKFrequent(nums, k):
    cnt = Counter(nums)
    buckets = [[] for _ in range(len(nums) + 1)]
    for x, c in cnt.items():
        buckets[c].append(x)
    res = []
    for c in range(len(buckets) - 1, 0, -1):
        for x in buckets[c]:
            res.append(x)
            if len(res) == k:
                return res
```

Complexity: O(n) time, O(n) space.

Median Finder (LC 295)

Question: `addNum`, `findMedian` on a stream. **Idea:** Max-heap `lo` (negated) + min-heap `hi`; keep `len(lo)` ≥ `len(hi)` and differ by ≤ 1.

```
import heapq

class MedianFinder:
    def __init__(self):
        self.lo, self.hi = [], []

    def addNum(self, num):
        heapq.heappush(self.lo, -num)
        heapq.heappush(self.hi, -heapq.heappop(self.lo))
        if len(self.hi) > len(self.lo):
            heapq.heappush(self.lo, -heapq.heappop(self.hi))

    def findMedian(self):
        if len(self.lo) > len(self.hi):
            return float(-self.lo[0])
        return (-self.lo[0] + self.hi[0]) / 2
```

Complexity: `addNum` O(log n), `findMedian` O(1).

Eliminate Maximum Monsters (LC 1921)

Question: `dist[i]`, `speed[i]`; weapon kills one per minute starting charged; lose if a monster arrives. Max kills? **Idea:** Arrival = ceil(dist/speed); kill in arrival order — the i-th shot at minute i loses if `arrival ≤ i`.

```
def eliminateMaximum(dist, speed):
    times = sorted((d + s - 1) // s for d, s in zip(dist, speed))
    for i, t in enumerate(times):
        if t <= i:
            return i
    return len(times)
```

Complexity: O(n log n) time, O(n) space.

Min Initial Health — gaming (min-heap of size rank)

Question (interpretation): Each level you fight the rank-th strongest opponent seen so far (new players join each level); each fight costs health equal to its strength. Find the minimum starting health so you survive all levels. **Idea:** Min-heap capped at `rank` tracks the strongest `rank` players; heap top is each level's opponent; total damage = sum of opponents.

```
import heapq

def min_initial_health(initial, levels, rank):
    heap, damage = [], 0

    def add(x):
```

```
        if len(heap) < rank:
            heapq.heappush(heap, x)
        elif x > heap[0]:
            heapq.heapreplace(heap, x)

    for x in initial:
        add(x)
    damage += heap[0]
    for new_players in levels:
        for x in new_players:
            add(x)
        damage += heap[0]
    return damage
```

Complexity: O((n + m) log rank) time, O(rank) space.

Backtracking

Sudoku: Validate (LC 36) ⭐ + Solve (LC 37) ⭐

Question: Validate: no digit repeats in any row/column/3×3 box. Solve: fill `.` cells so each row/column/box has 1–9. **Idea:** Sets per row/col/box, box index `(r//3)*3 + c//3`; solver = backtrack over empties with the same sets for O(1) checks.

```
def isValidSudoku(board):
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]
    for r in range(9):
        for c in range(9):
            v = board[r][c]
            if v == '.':
                continue
            b = (r // 3) * 3 + c // 3
            if v in rows[r] or v in cols[c] or v in boxes[b]:
                return False
            rows[r].add(v); cols[c].add(v); boxes[b].add(v)
    return True

def solveSudoku(board):
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]
    empties = []
    for r in range(9):
        for c in range(9):
            v = board[r][c]
            if v == '.':
```

```
                empties.append((r, c))
            else:
                rows[r].add(v); cols[c].add(v); boxes[(r // 3) * 3 + c // 3].add(v)

    def backtrack(i):
        if i == len(empties):
            return True
        r, c = empties[i]
        b = (r // 3) * 3 + c // 3
        for d in "123456789":
            if d in rows[r] or d in cols[c] or d in boxes[b]:
                continue
            board[r][c] = d
            rows[r].add(d); cols[c].add(d); boxes[b].add(d)
            if backtrack(i + 1):
                return True
            board[r][c] = '.'
            rows[r].remove(d); cols[c].remove(d); boxes[b].remove(d)
        return False

    backtrack(0)
```

Complexity: Validate O(1) (81 cells). Solve exponential worst case, fast via pruning; O(81) space.

Word Search (LC 79)

Question: Can `word` be built from adjacent grid cells (no reuse)? **Idea:** DFS from every cell; mark visited with `'#'`, restore on backtrack.

```
def exist(board, word):
    R, C = len(board), len(board[0])

    def dfs(r, c, i):
        if i == len(word):
            return True
        if r < 0 or c < 0 or r >= R or c >= C or board[r][c] != word[i]:
            return False
        tmp, board[r][c] = board[r][c], '#'
        ok = (dfs(r + 1, c, i + 1) or dfs(r - 1, c, i + 1) or
              dfs(r, c + 1, i + 1) or dfs(r, c - 1, i + 1))
        board[r][c] = tmp
        return ok

    return any(dfs(r, c, 0) for r in range(R) for c in range(C))
```

Complexity: O(R·C·3^L) time, O(L) recursion space.

Combination Sum (LC 39)

Question: Distinct candidates, reuse allowed; all unique combos summing to target. **Idea:** Backtrack with same index `i` for reuse; only move forward to avoid duplicate orderings.

```
def combinationSum(candidates, target):
    candidates.sort()
    res, path = [], []

    def dfs(start, remain):
        if remain == 0:
            res.append(path[:])
            return
        for i in range(start, len(candidates)):
            if candidates[i] > remain:
                break
            path.append(candidates[i])
            dfs(i, remain - candidates[i])
            path.pop()

    dfs(0, target)
    return res
```

Complexity: Exponential in the number of valid combinations.

Combination Sum II (LC 40)

Question: Candidates may repeat, each used at most once; unique combos only. **Idea:** Sort, recurse with `i + 1`, skip duplicates at the same depth (`i > start and c[i] == c[i-1]`).

```
def combinationSum2(candidates, target):
    candidates.sort()
    res, path = [], []

    def dfs(start, remain):
        if remain == 0:
            res.append(path[:])
            return
        for i in range(start, len(candidates)):
            if i > start and candidates[i] == candidates[i - 1]:
                continue
            if candidates[i] > remain:
                break
            path.append(candidates[i])
            dfs(i + 1, remain - candidates[i])
            path.pop()

    dfs(0, target)
    return res
```

Complexity: Exponential in the number of valid combinations.

Combination Sum III (LC 216)

Question: Exactly `k` distinct numbers from 1–9 summing to `n`. **Idea:** Backtrack digits 1–9, stop when `k` chosen.

```
def combinationSum3(k, n):
    res, path = [], []

    def dfs(start, remain):
        if len(path) == k:
            if remain == 0:
                res.append(path[:])
            return
        for d in range(start, 10):
            if d > remain:
                break
            path.append(d)
            dfs(d + 1, remain - d)
            path.pop()

    dfs(1, n)
    return res
```

Complexity: At most C(9, k) combinations.

Trie

WordDictionary (LC 211)

Question: `addWord`, `search` where `.` matches any letter. **Idea:** Trie; on `.` DFS every child.

```
class WordDictionary:
    def __init__(self):
        self.root = {}

    def addWord(self, word):
        node = self.root
        for ch in word:
            node = node.setdefault(ch, {})
        node['$'] = True

    def search(self, word):
        def dfs(i, node):
            if i == len(word):
                return '$' in node
            ch = word[i]
            if ch == '.':
```

```
                return any(dfs(i + 1, c) for k, c in node.items() if k != '$')
            return ch in node and dfs(i + 1, node[ch])
        return dfs(0, self.root)
```

Complexity: `addWord` O(L); `search` O(L) without dots.

Word Search II (LC 212)

Question: Find all words from a list formable in the grid. **Idea:** One trie for all words + one DFS per cell walking the trie; store the word at end nodes, prune exhausted branches.

```
def findWords(board, words):
    root = {}
    for w in words:
        node = root
        for ch in w:
            node = node.setdefault(ch, {})
        node['$'] = w

    R, C = len(board), len(board[0])
    res = []

    def dfs(r, c, parent):
        ch = board[r][c]
        node = parent[ch]
        if '$' in node:
            res.append(node.pop('$'))
        board[r][c] = '#'
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < R and 0 <= nc < C and board[nr][nc] in node:
                dfs(nr, nc, node)
        board[r][c] = ch
        if not node:
            parent.pop(ch)

    for r in range(R):
        for c in range(C):
            if board[r][c] in root:
                dfs(r, c, root)
    return res
```

Complexity: O(R·C·3^L) worst case, much faster with trie pruning.

Graph

Course Schedule (LC 207)

Question: Prerequisites `[a, b]` (b before a) — can all courses finish (no cycle)? **Idea:** Kahn's BFS topological sort; all processed ⟺ no cycle.

```
from collections import deque

def canFinish(numCourses, prerequisites):
    g = [[] for _ in range(numCourses)]
    indeg = [0] * numCourses
    for a, b in prerequisites:
        g[b].append(a)
        indeg[a] += 1
    q = deque(i for i in range(numCourses) if indeg[i] == 0)
    done = 0
    while q:
        u = q.popleft()
        done += 1
        for v in g[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return done == numCourses
```

Complexity: O(V + E).

Course Schedule II (LC 210)

Question: Return a valid course order, or `[]`. **Idea:** Same Kahn's, record the processing order.

```
from collections import deque

def findOrder(numCourses, prerequisites):
    g = [[] for _ in range(numCourses)]
    indeg = [0] * numCourses
    for a, b in prerequisites:
        g[b].append(a)
        indeg[a] += 1
    q = deque(i for i in range(numCourses) if indeg[i] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in g[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return order if len(order) == numCourses else []
```

Complexity: O(V + E).

Monster Min Cost — grid Dijkstra + path

Question: Grid where entering a cell costs its monster value (`-1` = wall); min total cost from start to goal, plus the path. **Idea:** Dijkstra on the grid; store a parent pointer on each relaxation, walk back from goal.

```
import heapq

def min_monster_cost(grid, start, goal):
    R, C = len(grid), len(grid[0])
    INF = float("inf")
    dist = [[INF] * C for _ in range(R)]
    parent = {}
    sr, sc = start
    dist[sr][sc] = grid[sr][sc]
    pq = [(grid[sr][sc], sr, sc)]
    while pq:
        d, r, c = heapq.heappop(pq)
        if d > dist[r][c]:
            continue
        if (r, c) == goal:
            break
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] >= 0:
                nd = d + grid[nr][nc]
                if nd < dist[nr][nc]:
                    dist[nr][nc] = nd
                    parent[(nr, nc)] = (r, c)
                    heapq.heappush(pq, (nd, nr, nc))
    if dist[goal[0]][goal[1]] == INF:
        return -1, []
    path, cur = [goal], goal
    while cur != start:
        cur = parent[cur]
        path.append(cur)
    return dist[goal[0]][goal[1]], path[::-1]
```

Complexity: O(RC log RC) time, O(RC) space. (If moves are only right/down, plain DP suffices.)

DP / Memo

Word Break (LC 139)

Question: Can `s` be split into dictionary words (reuse allowed)? **Idea:** `dp[i]` = prefix `s[:i]` segmentable; only check `j` within max word length.

```
def wordBreak(s, wordDict):
    words = set(wordDict)
    maxlen = max(map(len, words))
    dp = [False] * (len(s) + 1)
    dp[0] = True
    for i in range(1, len(s) + 1):
        for j in range(max(0, i - maxlen), i):
            if dp[j] and s[j:i] in words:
                dp[i] = True
                break
    return dp[-1]
```

Complexity: O(n·L) where L = max word length.

Min Days to Eat Oranges (LC 1553)

Question: Eat 1/day, or n/2 if divisible by 2, or 2n/3 if divisible by 3. Min days for n? **Idea:** Only halve / two-thirds matter; use single-eat days to reach divisibility: `f(n) = 1 + min(n%2 + f(n//2), n%3 + f(n//3))`, memoized.

```
from functools import lru_cache

def minDays(n):
    @lru_cache(None)
    def f(n):
        if n <= 1:
            return n
        return 1 + min(n % 2 + f(n // 2), n % 3 + f(n // 3))
    return f(n)
```

Complexity: O($\log^2 n$) states.

Combination Sum IV (LC 377)

Question: Count **ordered** sequences summing to target (orderings count separately). **Idea:** `dp[t] = Σ dp[t−x]`; target loop **outside** counts orderings (nums loop outside counts combinations).

```
def combinationSum4(nums, target):
    dp = [0] * (target + 1)
    dp[0] = 1
    for t in range(1, target + 1):
        for x in nums:
            if x <= t:
                dp[t] += dp[t - x]
    return dp[target]
```

Complexity: O(target · len(nums)) time, O(target) space.

Restaurant Value Meals — cheapest order ⭐ (Confluent)

Question: Menu items with prices + discounted bundles (value meals); find min price for a desired order (≤ 3 unique items in the original). **Idea:** State = remaining quantities tuple; try every option (single or bundle), recurse with memo. Skip options that don't shrink the state.

```
from functools import lru_cache

def best_price(menu, meals, order):
    items = list(order)
    idx = {it: i for i, it in enumerate(items)}
    target = tuple(order[it] for it in items)
    opts = []
    for it, p in menu.items():
        if it in idx:
            v = [0] * len(items)
            v[idx[it]] = 1
            opts.append((tuple(v), p))
    for bundle, p in meals:
        v = [0] * len(items)
        for it, q in bundle.items():
            if it in idx:
                v[idx[it]] = q
        if any(v):
            opts.append((tuple(v), p))

    @lru_cache(None)
    def dp(state):
        if all(q <= 0 for q in state):
            return 0.0
        best = float('inf')
        for vec, p in opts:
            nxt = tuple(max(0, s - b) for s, b in zip(state, vec))
            if nxt != state:
                best = min(best, p + dp(nxt))
        return best

    return dp(target)
```

Complexity: O(states × options); states = Π(qty+1).

Subset Sum — bank transactions

Question: Credits/debits, each used once — can any subset sum to target? E.g. `[7,-3,5,-2]`, target 4 → True (7 + −3). **Idea:** Iterative reachable-set DP; early exit when target appears.

```
def subset_sum(nums, target):
    reachable = {0}
    for x in nums:
        new_sums = {s + x for s in reachable}
        if target in new_sums:
            return True
        reachable |= new_sums
    return target in reachable
```

Complexity: O(n · S) time, O(S) space (S = distinct sums).

Warehouse Loading — target reach (bug fixed)

Question: Operations `W` (positive = load, negative = unload), executable in **any order**; does some ordering have a prefix summing exactly to target? **Idea:** Memoized DFS over (current load, used bitmask). ⚠️ The original snippet wrongly required `new_load > 0`, which forbids negative running totals — but its own sample needs order `[-1, 2, 4]`. Clarify with the interviewer whether negatives are allowed.

```
from functools import lru_cache

def can_reach_target(weights, target):
    n = len(weights)

    @lru_cache(None)
    def dfs(load, mask):
        if load == target:
            return True
        for i in range(n):
            if not mask & (1 << i):
                if dfs(load + weights[i], mask | (1 << i)):
                    return True
        return False

    return dfs(0, 0)
```

Complexity: O(n · 2^n) time and space. (If negatives disallowed, add a `load + w >= 0` guard.)

Wildcard Matching (LC 44)

Question: `?` matches one char, `*` matches any sequence. Does `s` match pattern `p`? **Idea:** Classic DP: `*` = skip it (`dp[i][j-1]`) or consume a char (`dp[i-1][j]`).

```
def isMatch(s, p):
    m, n = len(s), len(p)
    dp = [[False] * (n + 1) for _ in range(m + 1)]
    dp[0][0] = True
```

```
    for j in range(1, n + 1):
        if p[j - 1] == '*':
            dp[0][j] = dp[0][j - 1]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if p[j - 1] == '*':
                dp[i][j] = dp[i][j - 1] or dp[i - 1][j]
            elif p[j - 1] == '?' or p[j - 1] == s[i - 1]:
                dp[i][j] = dp[i - 1][j - 1]
    return dp[m][n]
```

Complexity: O(m·n) time and space.

Binary Search on Answer

Min Time — Hens & Grains (LC 2604)

Question: Hens and grains on a line; hens move 1 unit/s simultaneously. Min time to eat all grains? **Idea:** Binary search time `t`; feasibility: sweep hens left→right, each covers leftovers within reach (going left first costs double).

```
def minimumTime(hens, grains):
    hens.sort(); grains.sort()
    m = len(grains)

    def can(t):
        j = 0
        for h in hens:
            if j == m:
                return True
            if grains[j] < h:
                d = h - grains[j]
                if d > t:
                    return False
                reach = h + max(t - 2 * d, (t - d) // 2)
            else:
                reach = h + t
            while j < m and grains[j] <= reach:
                j += 1
        return j == m

    lo, hi = 0, 2 * (max(hens[-1], grains[-1]) - min(hens[0], grains[0]))
    while lo < hi:
        mid = (lo + hi) // 2
        if can(mid):
            hi = mid
        else:
```

```
            lo = mid + 1
    return lo
```

Complexity: O((n + m) log T).

Min Initial Strength vs Monsters (LC 4008)

Question: `monsters[i]` strengths; `boosts` add `v` to indices `[l, r]`. Start with strength s, fight left→right; win fight `i` if `s + bonus[i] ≥ monsters[i]`, then `s = max(0, s − monsters[i])`. Min starting s? **Idea:** More starting strength never hurts → binary search s; build bonuses with a difference array, simulate.

```
def minimumInitialStrength(monsters, boosts):
    n = len(monsters)
    diff = [0] * (n + 1)
    for l, r, v in boosts:
        diff[l] += v
        diff[r + 1] -= v
    bonus, cur = [0] * n, 0
    for i in range(n):
        cur += diff[i]
        bonus[i] = cur

    def can(s):
        for i in range(n):
            if s + bonus[i] < monsters[i]:
                return False
            s = max(0, s - monsters[i])
        return True

    lo, hi = 0, sum(monsters)
    while lo < hi:
        mid = (lo + hi) // 2
        if can(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo
```

Complexity: O(n + b + n log S), S = sum(monsters).

Intervals

Meeting Rooms II (LC 253)

Question: Min rooms for intervals `[start, end]`; end == start can share. **Idea:** Sort by start; min-heap of end times; reuse room if earliest end ≤ start.

```
import heapq

def minMeetingRooms(intervals):
    intervals.sort(key=lambda x: x[0])
    heap = []
    for s, e in intervals:
        if heap and heap[0] <= s:
            heapq.heappop(heap)
        heapq.heappush(heap, e)
    return len(heap)
```

Complexity: O(n log n) time, O(n) space.

Meeting Rooms III (LC 2402)

Question: `n` numbered rooms; each meeting takes the lowest free room, else waits (keeping duration); earliest original start has priority. Room with most meetings (lowest id on tie)? **Idea:** Two heaps: free (by id), busy (by end time). Release finished rooms first; if none free, delay the earliest-ending room.

```
import heapq

def mostBooked(n, meetings):
    meetings.sort()
    free = list(range(n))
    heapq.heapify(free)
    busy = []
    cnt = [0] * n
    for s, e in meetings:
        while busy and busy[0][0] <= s:
            _, r = heapq.heappop(busy)
            heapq.heappush(free, r)
        if free:
            r = heapq.heappop(free)
            heapq.heappush(busy, (e, r))
        else:
            end, r = heapq.heappop(busy)
            heapq.heappush(busy, (end + (e - s), r))
        cnt[r] += 1
    return cnt.index(max(cnt))
```

Complexity: O(m log m + m log n).

Meeting Scheduler (LC 1229)

Question: Two availability lists + duration → earliest `[start, start+duration]` free for both, else `[]`. **Idea:** Sort both; two pointers; overlap = [max starts, min ends]; advance the earlier-ending slot.

```
def minAvailableDuration(slots1, slots2, duration):
    slots1.sort(); slots2.sort()
    i = j = 0
    while i < len(slots1) and j < len(slots2):
        start = max(slots1[i][0], slots2[j][0])
        end = min(slots1[i][1], slots2[j][1])
        if end - start >= duration:
            return [start, start + duration]
        if slots1[i][1] < slots2[j][1]:
            i += 1
        else:
            j += 1
    return []
```

Complexity: O(n log n + m log m) time, O(1) extra space.

Linked Lists

Merge k Sorted Lists (LC 23)

Question: Merge k sorted linked lists into one. **Idea:** Min-heap of current heads; tuple `(val, i, node)` so nodes are never compared.

```
import heapq

def mergeKLists(lists):
    heap = [(node.val, i, node) for i, node in enumerate(lists) if node]
    heapq.heapify(heap)
    dummy = tail = ListNode()
    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next
```

Complexity: O(N log k) time, O(k) space. (`ListNode` is the standard LeetCode class.)

Streaming / Files

tail -n ⭐ (Confluent)

Question: Print the last `n` lines of a huge file in order, without loading it all. **Idea:** Streaming: `deque(maxlen=n)`. Seekable file: read fixed blocks **backwards** from the end, stop after > n newlines — memory scales with n, not file size.

```
from collections import deque

def tail_stream(lines, n):
    return list(deque(lines, maxlen=n)) if n > 0 else []

def tail_file(path, n, block=8192):
    if n <= 0:
        return []
    with open(path, "rb") as f:
        f.seek(0, 2)
        pos = f.tell()
        data = b""
        while pos > 0 and data.count(b"\n") <= n:
            step = min(block, pos)
            pos -= step
            f.seek(pos)
            data = f.read(step) + data
        lines = data.split(b"\n")
        if lines and lines[-1] == b"":
            lines.pop()                       # trailing newline artifact
        return [l.decode("utf-8", "replace") for l in lines[-n:]]
```

Complexity: Streaming O(file) time / O(n lines) space; backward version O(tail size) for both. (Thread-safety note: guard a shared deque with a lock if one thread reads while another snapshots.)

Design / Matching

Function Signature Matching (exact + variadic) ⭐

Question: Registry of functions `(name, param_types, is_card)`; given arg types, return all matches. Exact: same length, pairwise equal. Variadic (`is_card`): last param repeats 0+ times. **Idea:** Straightforward per-function check — fixed prefix must match, remaining args must equal the variadic type.

```
from dataclasses import dataclass
```

```
@dataclass(frozen=True)
class Function:
    name: str
    params: tuple
    is_card: bool = False

class FunctionRegistry:
    def __init__(self):
        self.functions = []

    def register(self, name, params, is_card=False):
        if is_card and not params:
            raise ValueError("variadic needs a parameter type")
        self.functions.append(Function(name, tuple(params), is_card))

    def match(self, args):
        return [f for f in self.functions if self._matches(f, args)]

    @staticmethod
    def _matches(f, args):
        if not f.is_card:
            return list(f.params) == list(args)
        fixed, var = f.params[:-1], f.params[-1]
        if len(args) < len(fixed):
            return False
        return (tuple(args[:len(fixed)]) == fixed
                and all(a == var for a in args[len(fixed):]))
```

Complexity: O(n·m) per lookup (n functions, m args), O(n·m) storage.

RandomQueue ADT

Question: Queue where `dequeue` removes a **uniformly random** element (not FIFO). Parts: core ops, equality, thread safety, RLE-backed equality. **Idea:** Dynamic array; dequeue = pick random index, swap with last, pop. Equality = multiset (frequency maps). Lock around ops for thread safety.

```
import random
import threading

class RandomQueue:
    def __init__(self):
        self._items = []
        self._lock = threading.RLock()

    def enqueue(self, item):
        with self._lock:
            self._items.append(item)

    def dequeue(self):
        with self._lock:
```

```
            if not self._items:
                raise IndexError("dequeue from empty RandomQueue")
            i = random.randint(0, len(self._items) - 1)
            self._items[i], self._items[-1] = self._items[-1], self._items[i]
            return self._items.pop()

    def peek(self):
        with self._lock:
            if not self._items:
                raise IndexError("peek from empty RandomQueue")
            return random.choice(self._items)

    def size(self):
        with self._lock:
            return len(self._items)
```

Complexity: O(1) expected per op, O(n) space. Equality: compare `Counter`s (O(n)); RLE: aggregate counts per value across runs, then compare (O(runs + distinct)).

Async / Timing

Debounce (LC 2627) ⭐

Question: Return a debounced `fn`: each call delays execution by `t`; a new call cancels the pending one. **Idea:** Keep one timer; cancel and restart on every call.

```
import threading

def debounce(fn, t_seconds):
    timer = None
    lock = threading.Lock()

    def wrapper(*args, **kwargs):
        nonlocal timer
        with lock:
            if timer is not None:
                timer.cancel()
            timer = threading.Timer(t_seconds, fn, args, kwargs)
            timer.start()
    return wrapper
```

Complexity: O(1) per call. (JS version: `clearTimeout` + `setTimeout`.)

Promise Time Limit (LC 2637)

Question: Wrap async `fn` so it rejects with `"Time Limit Exceeded"` if it takes longer than `t` ms.
Idea: Race `fn` against a timeout — `asyncio.wait_for` in Python, `Promise.race` in JS.

```
import asyncio

def time_limit(fn, t_ms):
    async def wrapper(*args, **kwargs):
        try:
            return await asyncio.wait_for(fn(*args, **kwargs), timeout=t_ms / 1000)
        except asyncio.TimeoutError:
            raise Exception("Time Limit Exceeded")
    return wrapper
```

Complexity: O(1) overhead per call.